

NAME: KELLY ATIENO DIANGÁ

Assembly Language & Analogue-to-Digital Conversion

PART 1: ASSEMBLY LANGUAGE

After analysing the assembly code for the segment display, I was able to comment on what the assembly codes do. The commented code and description is shown below.

```
.text
.thumb
.thumb_func
.align      2
.globl sevenSegment
.type       sevenSegment, %function
.syntax unified

/* Description
This code displays the ten decimal numerals (0 - 9) and six hexadecimal
"letter digits" (A - F) of the
hexadecimal number system in the seven segment display on the Virtual
Panel*/
/* Entry - r1- Stores the current number being displayed in the 7 segment
display*/
/*Exit - r3- stores new number to be displayed*/

/*Purpose of .equ */
.equ GPIOC, 0x400FF080 // Declare base address
.equ PDOR_OFFSET, 0x00 // Declare offset - Make it applicable for
different ports
.equ SEG_MASK, (0x7F<<4) // Declare segment Mask
.equ SEG_OFFSET, 4 //Declare offset used to shift segment mask to the
left by 4

/* Analyse the code below and provide comments */
sevenSegment:
    push {lr} //Store the return address in the Stack
    ldr      r1,=GPIOC //load the base address(0x400FF080) in r1,r1 =
GPIOC base address
    adr      r2,table // get address of table-Point r2 at the start
of the table
    ldrb     r2,[r2,r0] //Table lookup
    ldr      r3,[r1,#PDOR_OFFSET]//Current PDOR value
    and     r3,#~SEG_MASK //Clear appropriate bits to change number
    lsl     r2,r2,#SEG_OFFSET// Shift r2 to the left by 4 bits
    orr     r3,r3,r2 // Put in new bits for next value
    str     r3,[r1,#PDOR_OFFSET]//Update PDOR
    pop {pc} //Load program counter back into the CPU registers
    table: .byte
0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x67, //Table with decimal
numbers 0-9
    .end
```

PART 2: ANALOGUE-TO-DIGITAL CONVERTER (ADC)

To setup the ADC module to read the voltage value from the Analog knob of the ‘Virtual Panel’, I did the following ;

Adding header Files

To access the registers and declare suitable macros that will map to the registers in C program, I added the “derivative.h” header file. To test out the program by printing out the numbers in the termite, I included the header files “clock.h” and “stdio.h”.

```
#include "derivative.h"
#include<stdio.h>
#include "clock.h"
```

Defining Port Peripherals

This program used 7 port C pins as output to display the segment. These were defined by the SEG_MASK. I set the ADC module to read the voltage value from the potentiometer output from PTC0 (ADC0_SE14).

```
#define SEG_MASK (0x7F<<4)
#define ADC_channel 14
```

Define the minimum and Maximum display values that will correspond to the ADC minimum and Maximum voltage readings

I did some scaling for the conversion by defining the minimum values of the segment display that corresponded to the minimum voltage reading in the ADC channel. I set the minimum value to 0 and the maximum value as 9.

```
#define SEG_MIN 0
#define SEG_MAX 9
```

Define the ADC Range

I set the ADC range to a 10-bit resolution for more accurate results by reducing the quantization error.

```
#define ADC_MAX ((1<<10)-1)//1023
```

Functions

I created two functions as shown below;

1. Delay Function

I used this function to create some delay so as to be able to see the changes taking place as the analogue to digital conversion occurs for numbers as well as the hex letters.

```
void delay()
{
    int delayCount;
```

```

    for(delayCount = 0; delayCount < 1000000; delayCount++)
    {
        asm("nop"); // No - operation but don't delete
    }
}

```

2. Function for initialization of ADC

This function contains the ADC clock which is enabled so as to use any of the ADC registers.

I also configured the ADC channel here by setting it to low power operation(restrict maximum clock speed) with a conversion resolution of 10 bits.

```

void ADC_initialise()
{
    // Enable clock to ADC
    SIM_SCGC6 |= SIM_SCGC6_ADC0_MASK;
    // Set to Low Power and 10-bit conversion
    ADC0_CFG1 = ADC_CFG1_ADLPC_MASK | ADC_CFG1_MODE(2);
}

```

3. Function for conversion of ADC

This is where the conversion from voltage to decimal numerals (0 - 9) and six hexadecimal "letter digits" (A - F) occurs. I used ADC0_SC1A status and control register to start the conversion process and used a while loop to allow for the conversion process to finish. This function then returns an unsigned integer value of either decimal number or letter digit.

```

unsigned int ADC_convert()
{
    //Trigger conversion on ADC Channel 14
    ADC0_SC1A = ADC_SC1_ADCH(ADC_channel);

    //Wait for conversion to complete
    while((ADC0_SC1A & ADC_SC1_COCO_MASK)==0);

    // store result from ADC conversion into variable
    return SEG_MIN + ((unsigned int)ADC0_RA * (SEG_MAX-SEG_MIN))/ADC_MAX;
}

```

Main Function

Enable clock to ADC and Ports

The clock for Port C which acts as a master switch is enabled in the main function. I also called the ADC initialisation function here.

```

ADC_initialise();
clock_initialise();
// Enable clock to PORT C
SIM_SCGC5 |= SIM_SCGC5_PORTC_MASK;

```

Configure Port Pins

I made port C pin 4-10 simple port pins with a high drive strength.

```
PORTC_PCR4 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
PORTC_PCR5 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
PORTC_PCR6 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
PORTC_PCR7 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
PORTC_PCR8 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
PORTC_PCR9 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
PORTC_PCR10 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
```

Set Direction of Pins

I then set the port direction for SEG_MASK-port C pin 4-10, as outputs

```
GPIOC_PDDR |= SEG_MASK;
```

Conversion and Display

I called the ADC conversion function here and used a for loop to run the function infinitely. I also displayed the results on the termite and called the delay function here so as to observe the changes in decimal numbers.

```
for(;;)
{
    unsigned counter = ADC_convert();
    printf("ADC VALUE = %d\n", counter);
    delay();
}
```

FULL CODES

```
/*
 * main implementation: use this 'C' sample to create your own application
 */
#include "derivative.h" /* include peripheral declarations */
#include <stdio.h>
#include "clock.h"

#define SEG_MASK (0x7F<<4)
#define ADC_channel 14

//Minimum and Maximum value for segment display
#define SEG_MIN 0
#define SEG_MAX 9

//Range of ADC values(ADC resolution-10bit)
#define ADC_MAX ((1<<10)-1)//1023

//Declare sevenSegment as global variable
extern unsigned int sevenSegment(unsigned int value);

// Create delay Function
void delay()
{
    int delayCount;
```

```

    for(delayCount = 0; delayCount < 1000000; delayCount++)
    {
        asm("nop"); // No - operation but don't delete
    }
}
//
void ADC_initialise()
{
    // Enable clock to ADC
    SIM_SCGC6 |= SIM_SCGC6_ADC0_MASK;
    // Set to Low Power and 10-bit conversion
    ADC0_CFG1 = ADC_CFG1_ADLPF_MASK | ADC_CFG1_MODE(2);
}
unsigned int ADC_convert()
{
    //Trigger conversion on ADC Channel 14
    ADC0_SC1A = ADC_SC1_ADCH(ADC_channel);

    //Wait for conversion to complete
    while((ADC0_SC1A & ADC_SC1_COCO_MASK)==0);

    // store result from ADC conversion into variable
    return SEG_MIN + ((unsigned int)ADC0_RA * (SEG_MAX-SEG_MIN))/ADC_MAX;
}
}
int main(void)
{
    ADC_initialise();
    clock_initialise();
    // Enable clock to PORT C
    SIM_SCGC5 |= SIM_SCGC5_PORTC_MASK;
    // Make Port C.4-10 pin a simple port pin with high drive strength
    PORTC_PCR4 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR5 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR6 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR7 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR8 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR9 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR10 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;

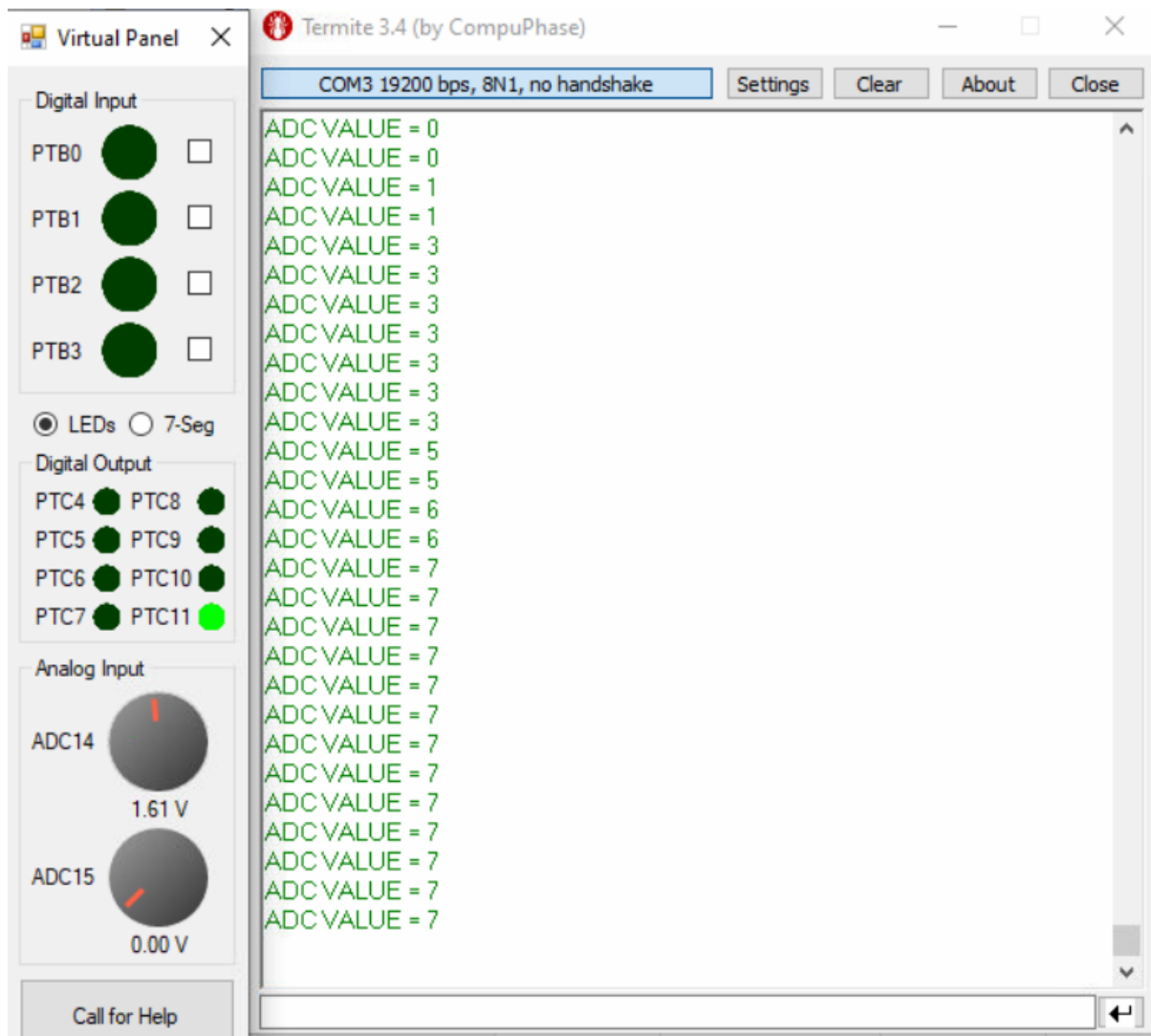
    // Make Port C.4-10 an output
    GPIOC_PDDR |= SEG_MASK;

    for(;;)
    {
        unsigned counter = ADC_convert();
        printf("ADC VALUE = %d\n",counter);
        delay();
    }
    return 0;
}

```

RESULTS

The following are the conversion results for the codes shown above as shown through the serial console(Termite).



PART 3: APPLICATION & INTEGRATION

I modified the given assembly program in Part 1 to make it display hex value from 0 – F and map the ADC reading to the seven segment display to make it display (min value -> 0, max value -> F) with gradual change.

MODIFICATIONS MADE

1. Changing the table lookup to 15

In the assembly code, I adjusted the table lookup to display decimal numbers 0-9 and six hexadecimal "letter digits" (A - F). I did this as shown below;

table: .byte

0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x67, 0x77, 0x7C, 0x39, 0x5E, 0x79,
0x71

2. Removing header files

For this part of the lab, I used the 7-segment display from the virtual panel instead of the termite to print out the digits. I therefore removed the header files "clock.h" and <stdio.h> that would have otherwise been used to write to the termite. The only header file used was thus;

```
#include "derivative.h"
```

3. Adjusting Maximum Value to 15

Initially my maximum value was 9. However, because I also wanted to display the six hexadecimal "letter digits" (A - F), I changed the maximum value to 15.

```
#define SEG_MAX 15
```

4. Declaring sevenSegment a global function

To implement the assembly code in C program, I needed to make it global, this is done using the code below.

```
extern unsigned int sevenSegment(unsigned int value);
```

5. Making use of Port C

I used port C pins 4-10 as outputs to light up the seven segment display. This is done using the SEG_MASK as shown below.

```
GPIOC_PDDR |= SEG_MASK;
```

6. Using sevenSegment function for display

Instead of using the termite, I used the 'Virtual Panel' after switching the output mode to '7-Seg' output. I therefore removed the printf function and instead used the sevenSegment function with its input as the converted value from the ADC conversion function. I still used the for loop to let the program run infinitely.

```
for(;;)
{
    unsigned segNumber = ADC_convert();

    sevenSegment(segNumber);
    delay();
}
```

The code below shows the modified code.

ASSEMBLY CODE

```
        .text
        .thumb
        .thumb_func
        .align      2
        .globl sevenSegment
        .type        sevenSegment, %function
        .syntax unified

/* Description
```

```

This code displays the ten decimal numerals (0 - 9) and six hexadecimal
"letter digits" (A - F) of the
hexadecimal number system in the seven segment display on the Virtual
Panel*/
/* Entry - r1- Stores the current number being displayed in the 7 segment
display*/
/*Exit - r3- stores new number to be displayed*/

/*Purpose of .equ */
.equ GPIOC, 0x400FF080 // Declare base address
.equ PDOR_OFFSET, 0x00 // Declare offset - Make it applicable for
different ports
.equ SEG_MASK, (0x7F<<4) // Declare segment Mask
.equ SEG_OFFSET, 4 //Declare offset used to shift segment mask to the
left by 4

/* Analyse the code below and provide comments */
sevenSegment:
    push {lr} //Store the return address in the Stack
    ldr r1,=GPIOC //load the base address(0x400FF080) in r1,r1 =
GPIOC base address
    adr r2,table // get address of table-Point r2 at the start
of the table
    ldrb r2,[r2,r0] //Table lookup
    ldr r3,[r1,#PDOR_OFFSET]//Current PDOR value
    and r3,#~SEG_MASK //Clear appropriate bits to change number
    lsl r2,r2,#SEG_OFFSET// Shift r2 to the left by 4 bits
    orr r3,r3,r2 // Put in new bits for next value
    str r3,[r1,#PDOR_OFFSET]//Update PDOR
    pop {pc} //Load program counter back into the CPU registers
    table: .byte
0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x67,0x77,0x7C,0x39,0x5E,0x79,
0x71//Table with decimal numbers 0-9 and six hex letters A-F
    .end

```

MAIN CODE

```

/*
 * main implementation: use this 'C' sample to create your own application
 */
#include "derivative.h" /* include peripheral declarations */

#define SEG_MASK (0x7F<<4) //Use port C pin 4-10
#define ADC_channel 14 //Use ADC Channel 14

//Minimum and Maximum value for segment display
#define SEG_MIN 0
#define SEG_MAX 15

//Range of ADC values(ADC resolution-10bit)
#define ADC_MAX ((1<<10)-1)//1023

//Declare sevenSegment assembly code as global
extern unsigned int sevenSegment(unsigned int value);

```



```

// Create delay Function
void delay()
{
    int delayCount;
    for(delayCount = 0; delayCount < 1000000; delayCount++)
    {
        asm("nop"); // No - operation but don't delete
    }
}
//
void ADC_initialise()
{
    // Enable clock to ADC
    SIM_SCGC6 |= SIM_SCGC6_ADC0_MASK;
    // Set to Low Power and 10-bit conversion
    ADC0_CFG1 = ADC_CFG1_ADLPFC_MASK | ADC_CFG1_MODE(2);
}
unsigned int ADC_convert()
{
    //ADC_initialise();
    //Trigger conversion on ADC Channel 14
    ADC0_SC1A = ADC_SC1_ADCH(ADC_channel);

    //Wait for conversion to complete
    while((ADC0_SC1A & ADC_SC1_COCO_MASK) == 0);

    // store result from ADC conversion into variable
    return SEG_MIN + ((unsigned int)ADC0_RA * (SEG_MAX-SEG_MIN))/ADC_MAX;
}
int main(void)
{
    ADC_initialise();
    // Enable clock to PORT C
    SIM_SCGC5 |= SIM_SCGC5_PORTC_MASK;
    // Make Port C.4-10 pin a simple port pin with high drive strength
    PORTC_PCR4 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR5 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR6 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR7 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR8 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR9 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;
    PORTC_PCR10 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;

    // Make Port C.4-10 an output
    GPIOC_PDDR |= SEG_MASK;

    for(;;)
    {
        unsigned segNumber = ADC_convert();

        sevenSegment(segNumber);
        delay();
    }

    return 0;
}

```

VIDEO DEMONSTRATION

To view the output, I opened the 'Virtual Panel' in the Remote Lab and switched the output mode to '7-Seg' output. The following link shows the demonstration for the above source code.

<https://youtu.be/AZB6sM28gyw>